01DAS

Intro lecture

```
old_name = "01DAS"
new_name = old_name[:2] + old_name[3] + old_name[2] +
old_name[4:]
print(new_name)
```

01ADS

Intro lecture

Why this course had to exist

TEN YEARS AGO

The curriculum had no place where students built a real software product in a team.

THE GAP

Python, Git, collaboration and product thinking had to be learned together - while turning ML / DL ideas into a working system.

01ADS was created to close that gap.

The course is evolving again

AI changes how software gets built - not why engineering matters.

YOU WILL STILL

Deliver a real software project using ML, DL and LLMs.

WHAT CHANGES

AI-assisted development, vibe coding and much faster prototyping.

WHAT STAYS

Ownership, teamwork, validation, maintainability and a working product.

What students consistently told us (2019-2025)

REAL-WORLD VALUE

Practical skills transferred directly to work and future projects.

LEARNING THAT STICKS

Students kept using the knowledge long after the course ended.

TEAM EXPERIENCE

The project felt like real data science work, not a classroom exercise.

DEMANDING

The workload was high and required substantial independent learning.

TOO FEW CREDITS

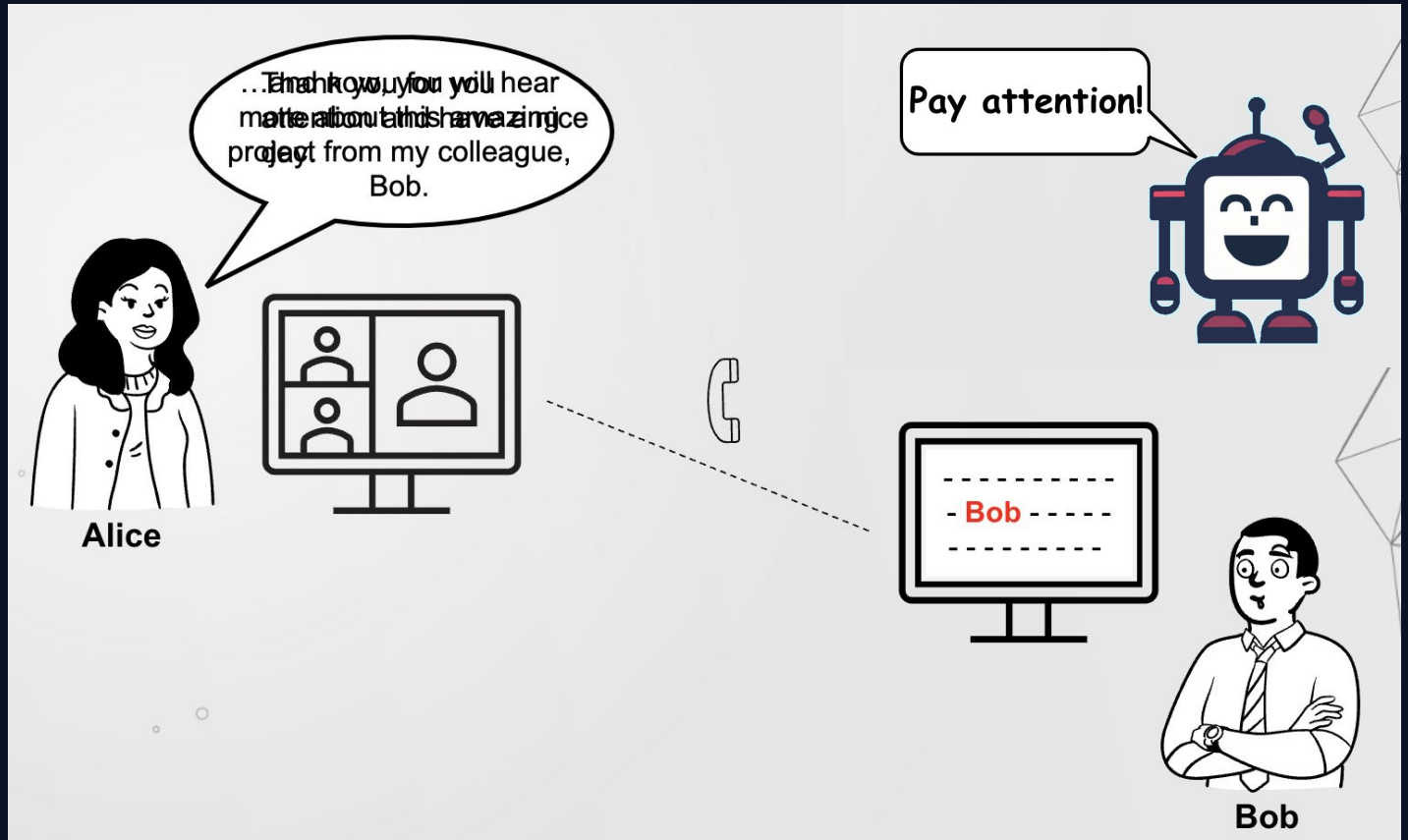
The effort was consistently judged greater than the credit allocation.

WORTH IT

Despite the workload, students strongly recommended the course.

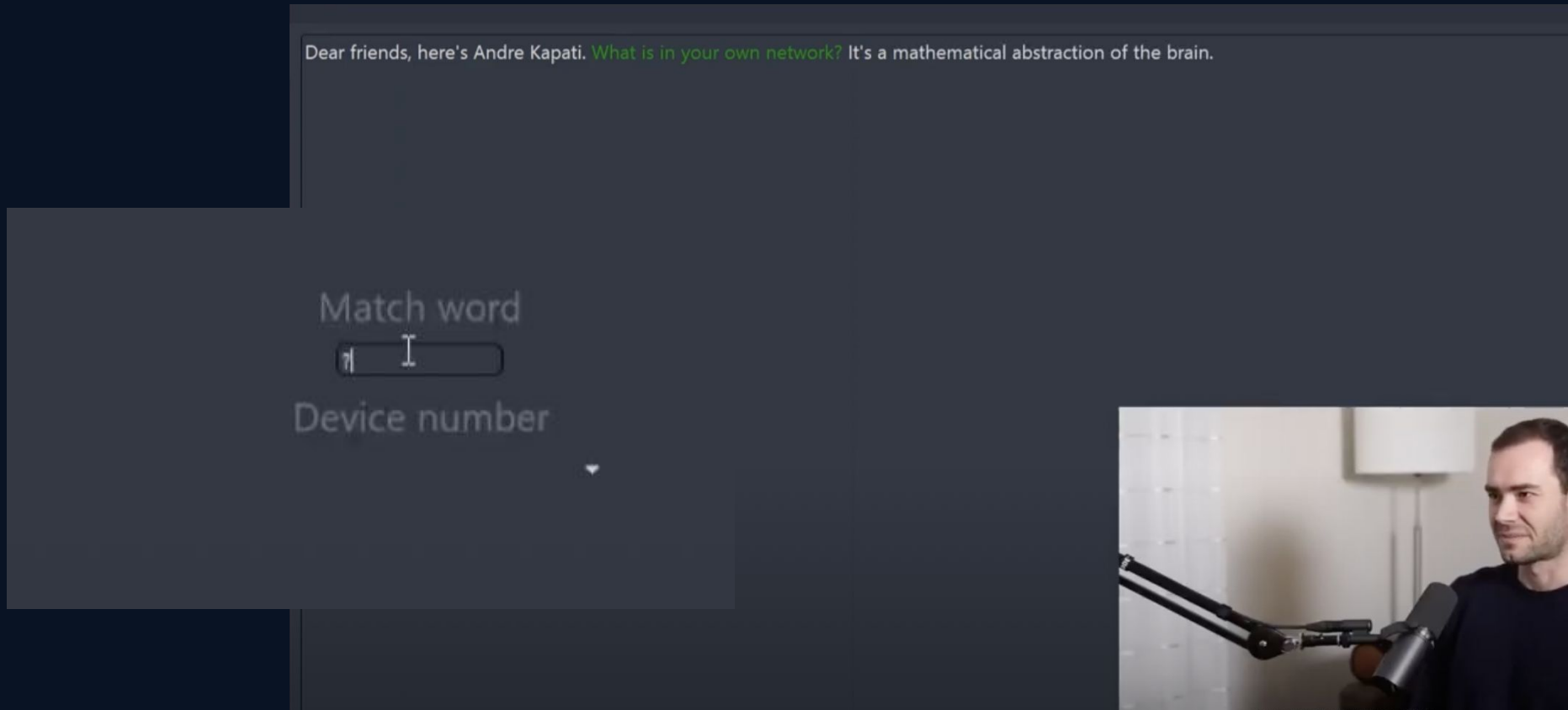
Projects from previous years

Call-Out Bot

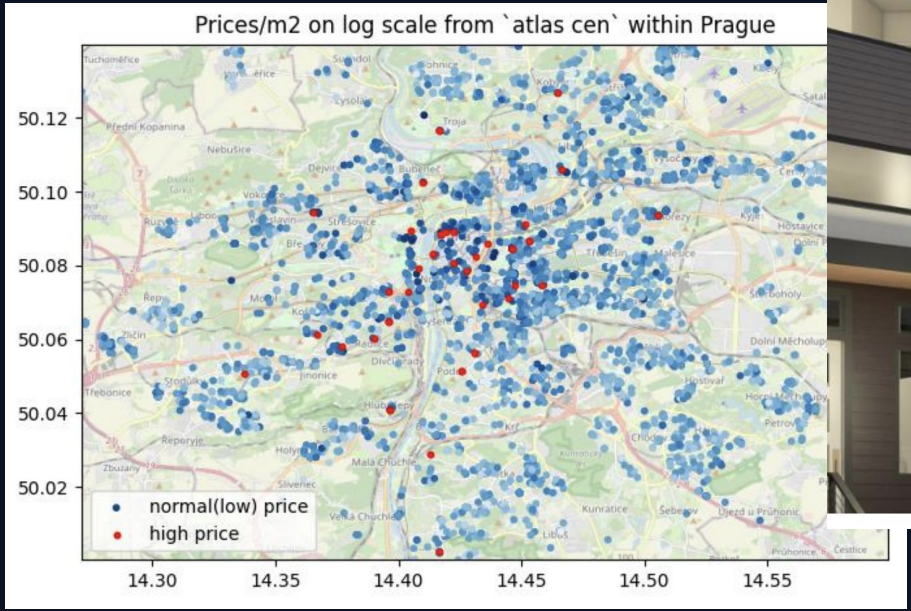

…Thank you, for you will hear
matter about this amazing
project from my colleague,
Bob.
Alice
Pay attention!
Bob
Bob

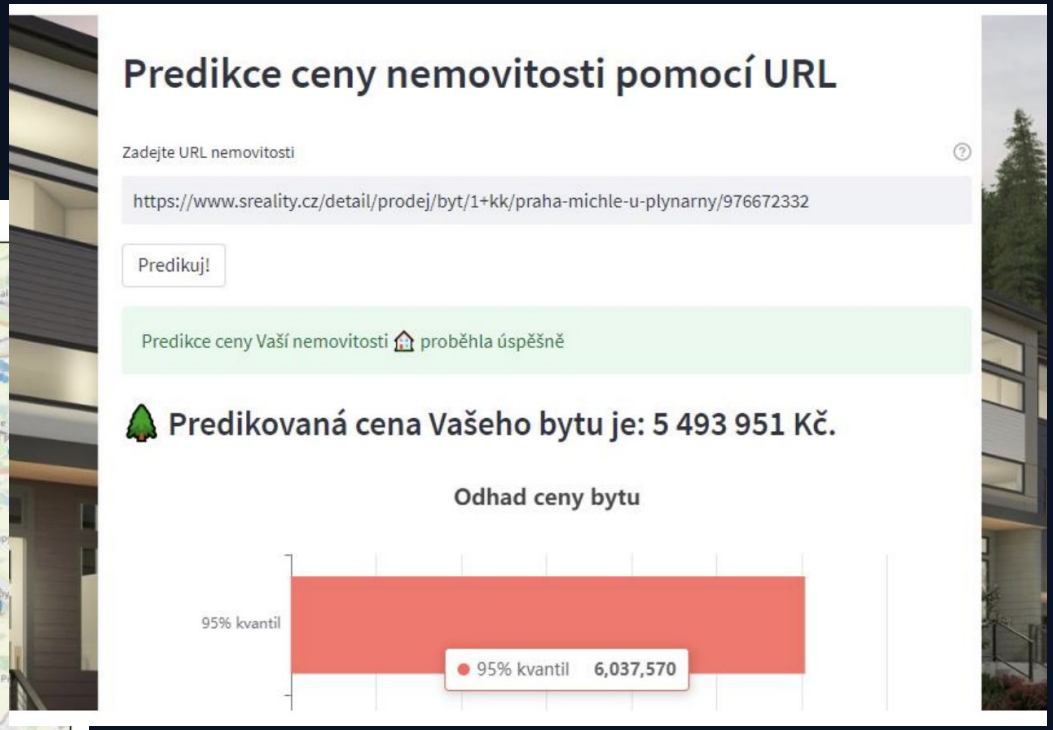
Call-Out Bot (2022)

https://www.youtube.com/watch?v=LB9bIARP3QI

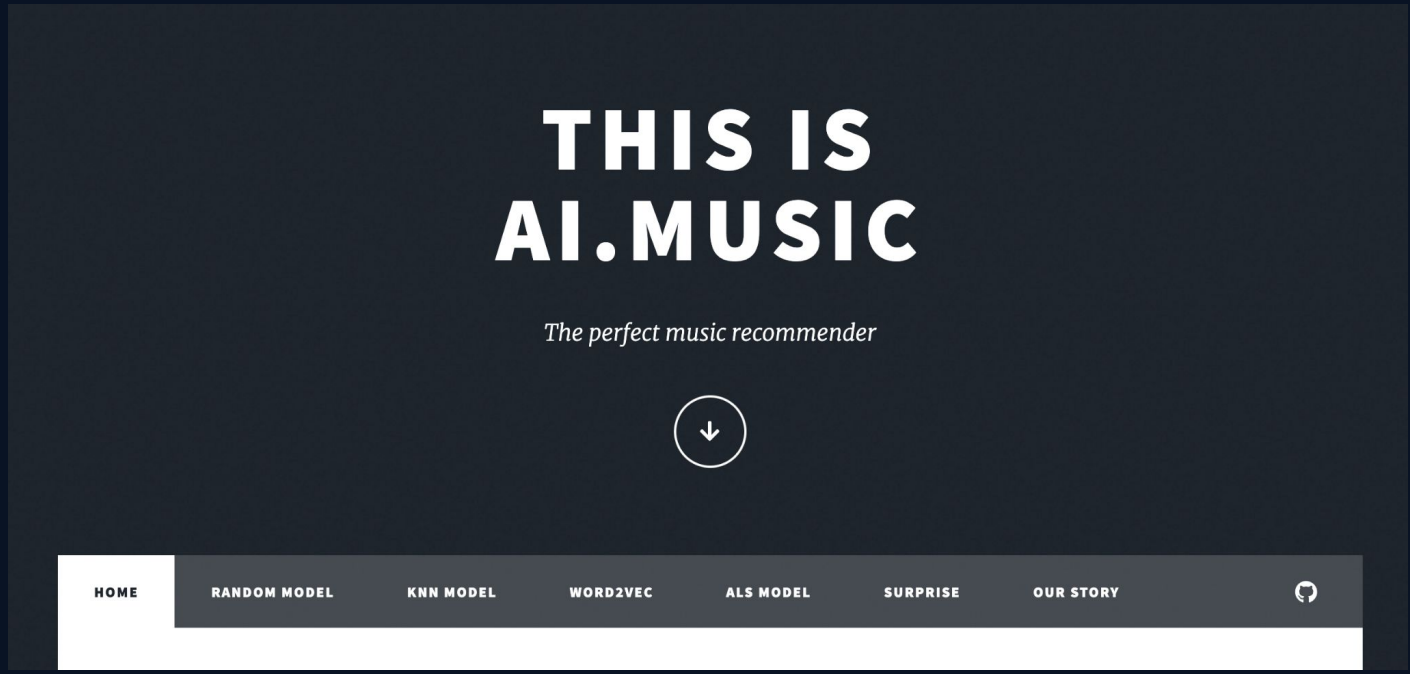


Real Estate (2022)

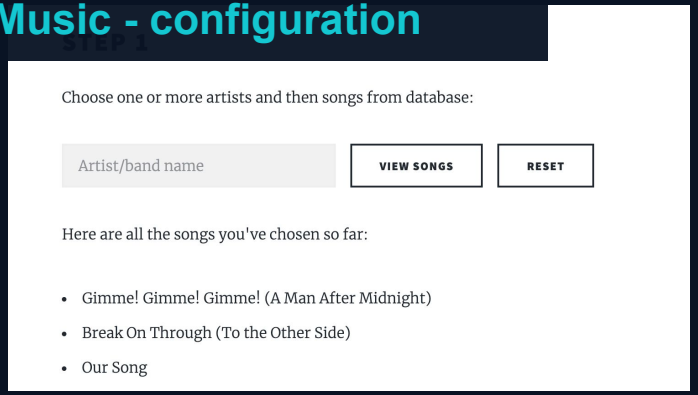


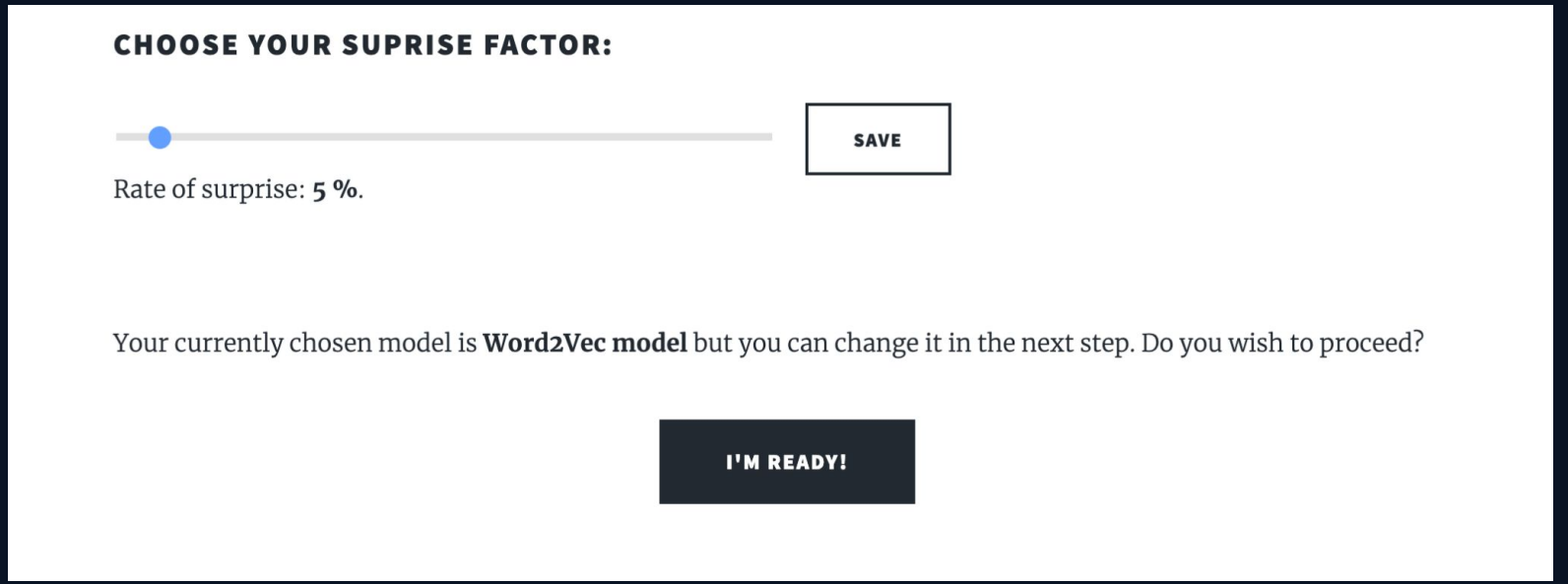


Song Recommender - AI.Music (2021)



AI.Music - configuration

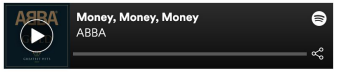




AI.Music - recommendations

If you want to get more (and hopefully better) recommendations simply click the ❤ icon. This will add songs you like to your list and you can recalculate your recommendations to get even more amazing songs tailored just for you. You can also dislike a song with 🚫.

ABBA: Money, Money, Money



ABBA: Waterloo

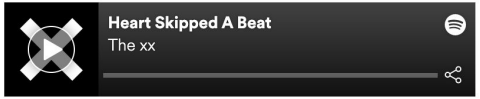


ABBA: Chiquitita



Rihanna: S&M

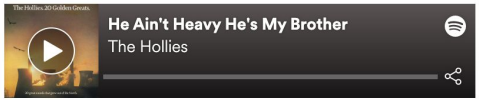




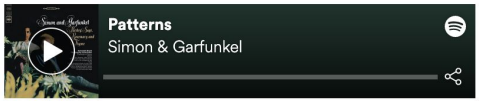
Blondie: Heart Of Glass



The Hollies: He Ain't Heavy He's My Brother



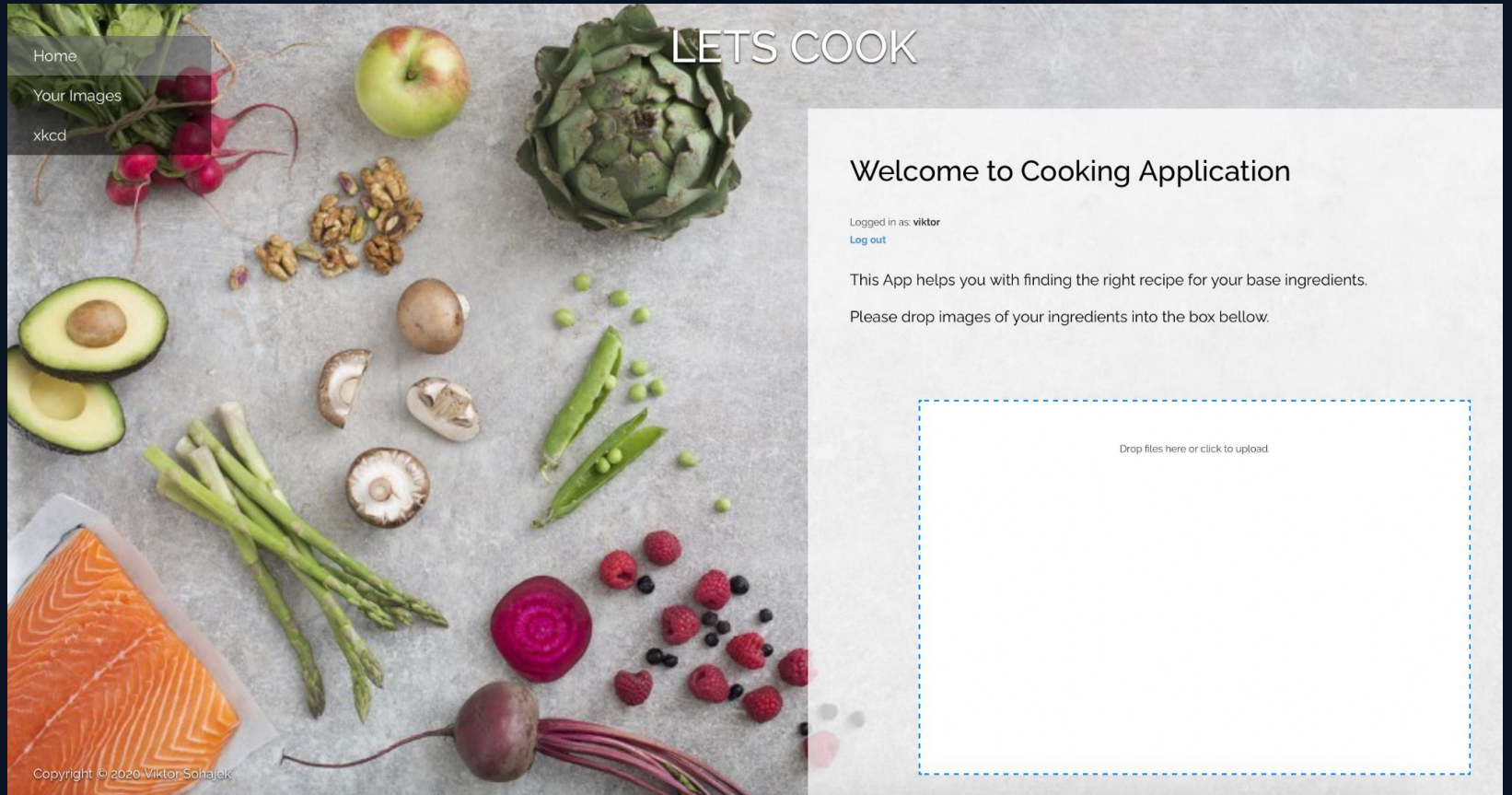
Simon & Garfunkel: Patterns



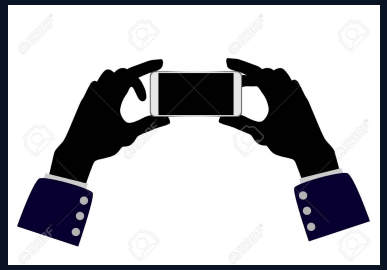
The Gibson Brothers: Que Sera Mi Vida



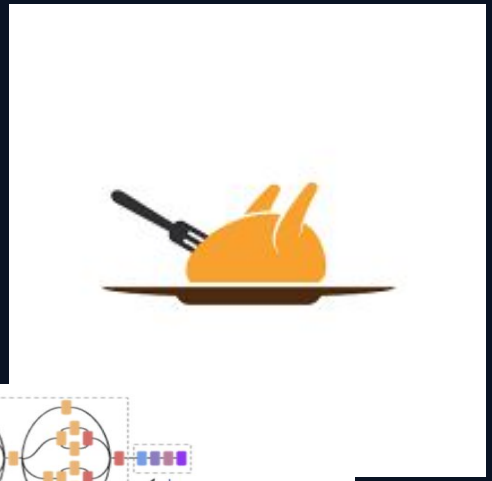
Cooking App (2021)

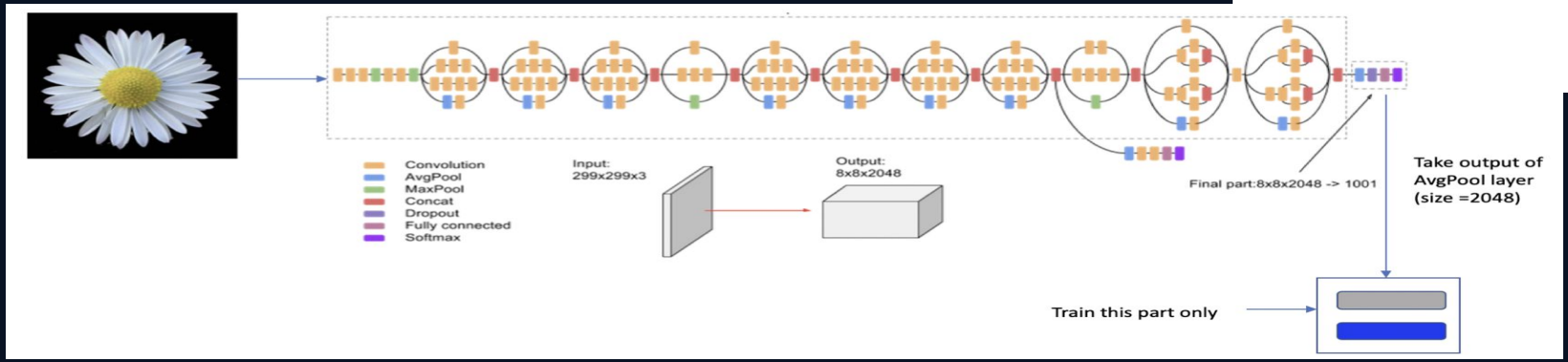


Take a photo of ingredients in your fridge, choose a diet, and start cooking!

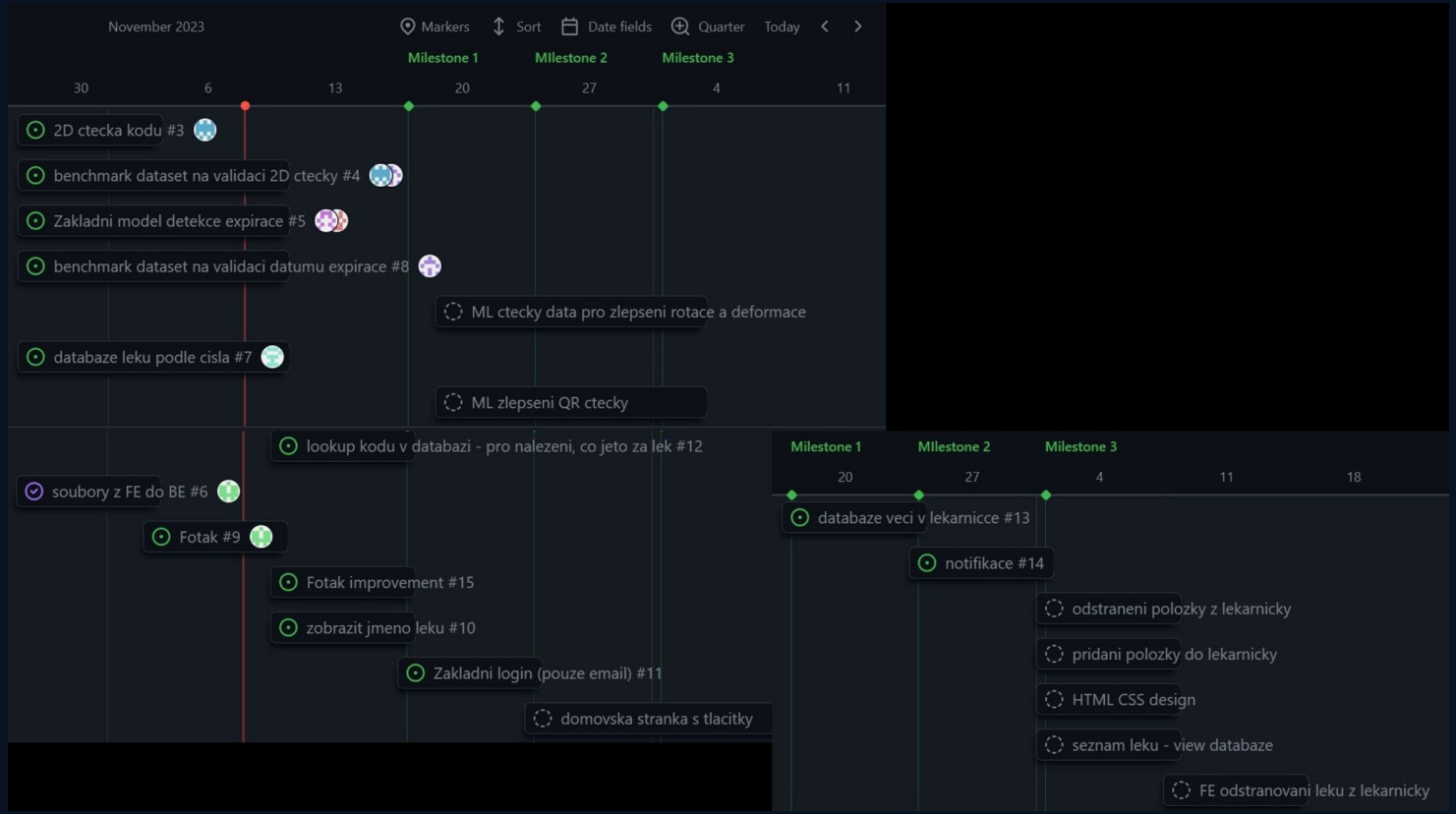








Medicine Cabinet



Political Confession

Do politicians bend the truth?
Change their views with the polls?
Avoid uncomfortable debates?

Political Confession lets you interview a simulated politician - or moderate a debate between opposing sides.

The system surfaces changes of position, past mistakes and contradictions - making political claims easier to explore and challenge.



GPT Sell


Chtel bych novej notebook, ne moc drahej.
No hlavně by měl být růžovej a tenkej. Do kabelky.
Vsechno mame! Kolik ma mit RAM?
OPEN
Rozumim... Něco bych tu pro vás měl.

GPT Sell - Technology

LLMs

LangChain

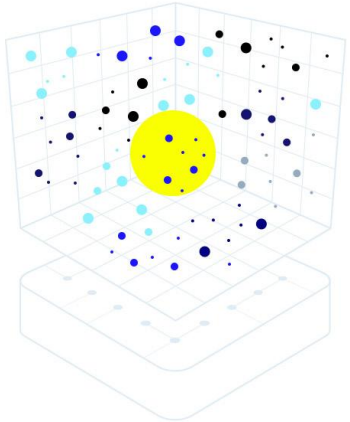
Vector databases

LlamaIndex

Agentic systems









How this course works

Work in a team of about five with clear roles.

Build one assessed product for a real stakeholder.

Ship continuously - small weekly progress beats an exam-period sprint.

Use Python, Git and AI coding tools responsibly.

Communicate decisions, risks and results.

Learn by building

We throw you into the deep end - and teach you to swim by building.

BUILD

Start with a real software project in week 1.

LEARN

Pick up Python, Git and ML / DL / LLM concepts when the project demands them.

SHIP

Use AI tools, test assumptions and deliver a working product.

The project is the curriculum.

Semester roadmap

W01 - Kick-off, teams, project selection, product setup

W02 - Development environment, Git and Python tooling

W03-08 - Topic sessions, team demos and iterative delivery

W09-11 - Integration, testing and final build

W13 - Demo Day, presentation and evaluation

TEAM SETUP

One team. One product.

Five students. Shared ownership. A real system to ship.

Know your starting point

Score yourself from 0 to 3. This is not graded.

0	1	2	3
Never tried	Tried with help	Can work independently	Can teach or review

We will use the results to distribute work, pair people and identify learning goals - not to rank the team.

Skills self-check

PYTHON & DATA

CLI scripts, Jupyter, pandas/NumPy, SQL, APIs

GIT & COLLABORATION

branches, pull requests, reviews, merge conflicts

ML & RAG

embeddings, vector search, evaluation, LLM APIs

DELIVERY

tests, Docker, Linux/SSH, deployment, monitoring

Mark each item 0-3. Pick one skill you want to strengthen this semester.

Team 01 - FJFI RAG

Guliev Ruslan

Chrastina Jan

Paleta Vojtěch

Stránský Vojtěch

Artímez Peña Aldo

One team, one shared backlog, one final product.

How we will work

TEAM LEAD
coordination, planning, blockers

PRODUCT & STAKEHOLDER
backlog, decisions, user value

RAG & BACKEND
ingestion, retrieval, APIs

DATA & EVALUATION
datasets, citations, quality

FRONTEND & DEVOPS
UI, deployment, monitoring

Roles are starting points. Everyone codes, reviews and documents.

Access checklist

1. Send your GitHub username
2. Generate an SSH key and share the public key only
3. Get added to FJFI/fjfi-rag
4. Get server access and verify SSH
5. Clone the repository, run the stack and open your first PR

Never share a private SSH key.

THE PROJECT

rag.fjfi.cvut.cz

AI knowledge platform for FJFI CTU

What are we building?

SEARCH

Theses and reviews

Personal and shared documents

FJFI and CTU regulations

ASK

Grounded answers

Source citations

Transparent evidence

CONNECT

Web application

MCP for Claude and Codex

REST API

MVP phase. The staging server and base infrastructure are already running.

Starting point

API

v0.1.0, staging, running

DATABASE

PostgreSQL + pgvector, connected

INTERFACES

web, MCP and REST API

EXTENSIBILITY

ready for local-model integration

Your job: turn the working foundation into a useful, reliable product.

MVP roadmap

15 OCT

Login, thesis indexing, search and cited answers, MCP/API

29 OCT

Structured reviews, reviewer database and statistics

5 NOV

AI-assisted thesis review compared with human reviews

19 NOV

Personal OneDrive folders and shared group libraries

3 DEC

FJFI/CTU regulations with versions in force

17 DEC

Final presentation

Let's open the product

https://rag.fjfi.cvut.cz/

Explore the current MVP and roadmap.
Then we will turn the roadmap into a team backlog.

Repository: github.com/FJFI/fjfi-rag (internal)

PROJECT MANAGEMENT

Turn uncertainty into steady progress

Align the team, shorten feedback loops and keep shipping.

What project management really does

ALIGN
A shared outcome and clear priorities

FOCUS
The next valuable slice, not everything at once

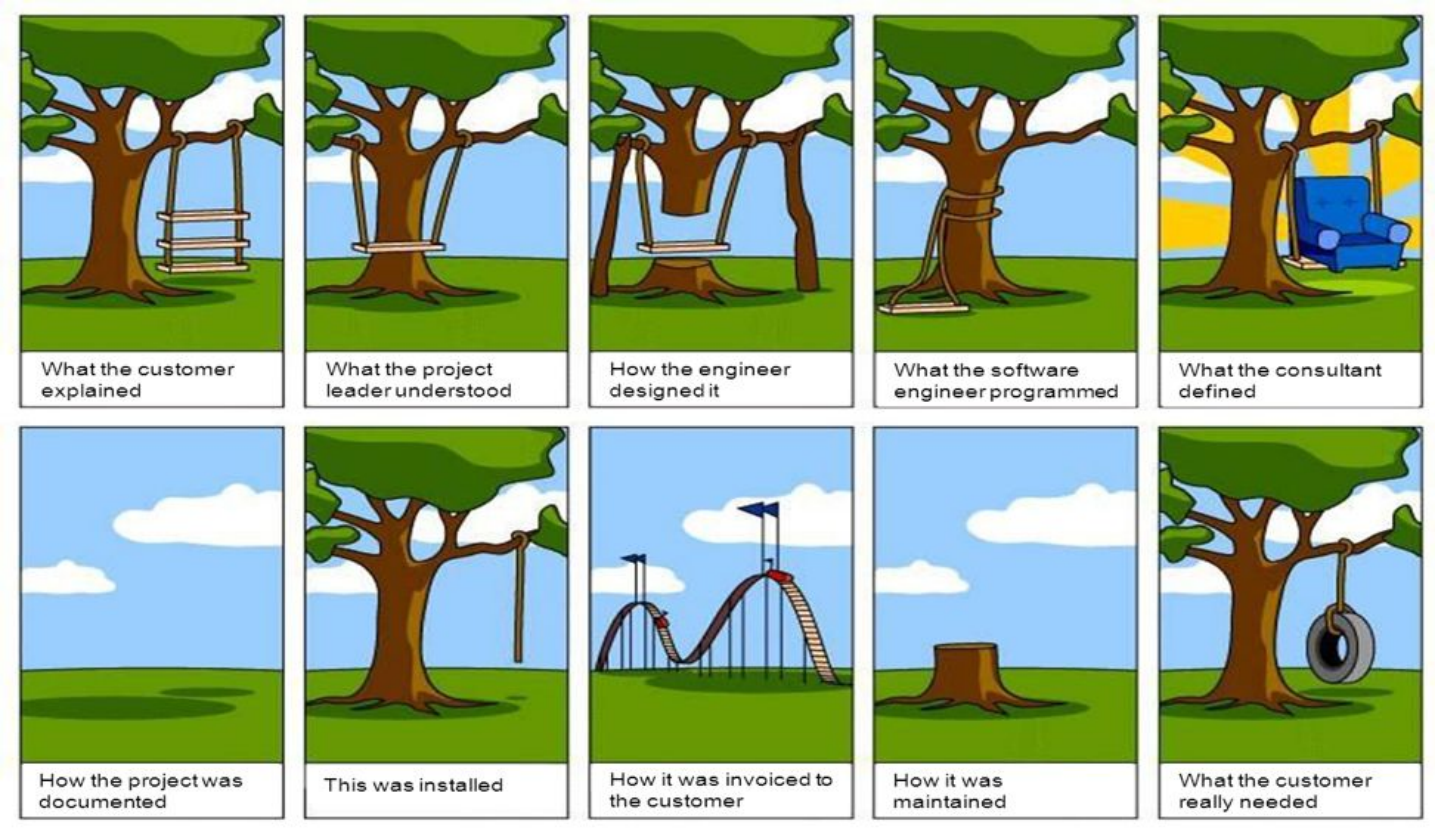
EXPOSE
Ownership, blockers, risks and decisions

LEARN
Frequent feedback from working software

ADAPT
Change the plan when evidence changes

It is not more meetings. It is less hidden work.

Why project management exists



Roles and decision rights

STAKEHOLDER / INSTRUCTOR

Defines value, priorities and acceptance.

Explains why the product matters.

TEAM LEAD

Coordinates delivery, resolves blockers and protects focus.

TEAM MEMBERS

Own issues, communicate progress, review changes and ship quality.

Discuss openly. Decide clearly. Commit as one team.

One source of truth

ROADMAP
Where are we going?

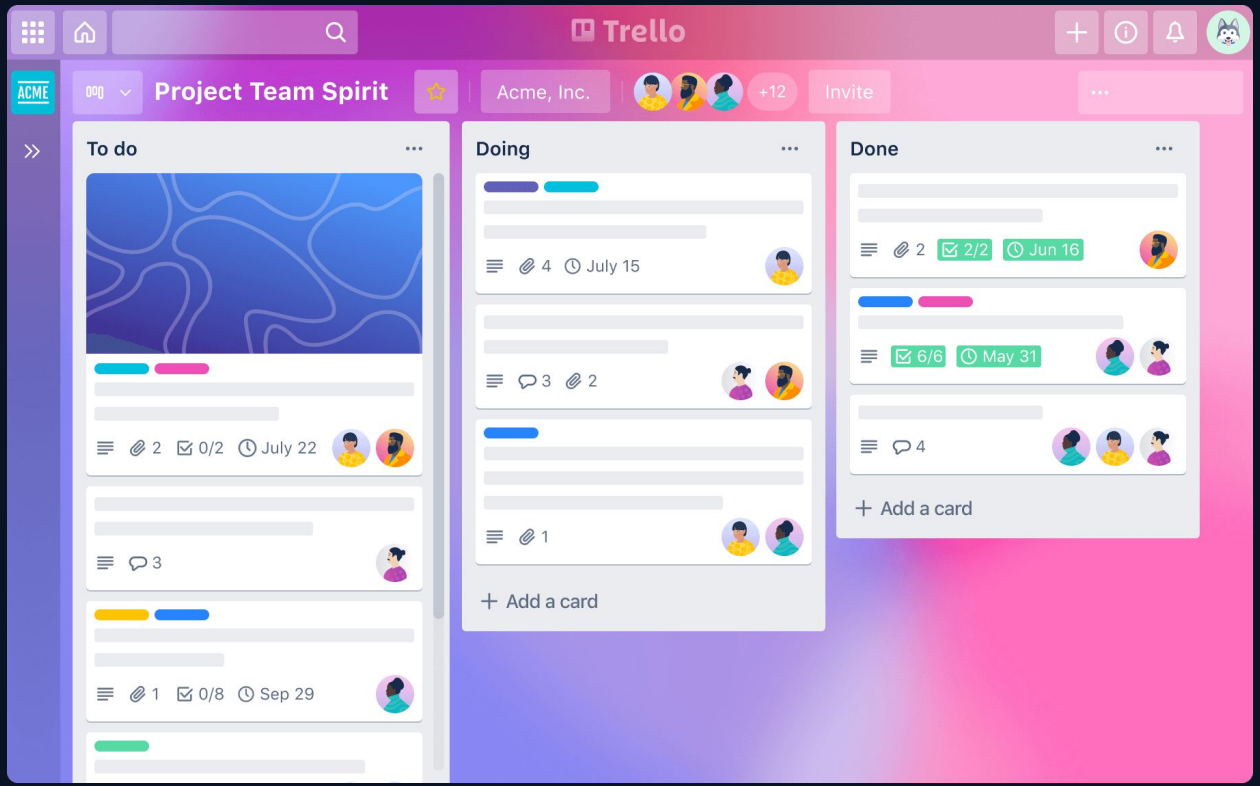
ISSUES
What should change and why?

PULL REQUESTS
How did the product change?

DOCS + ADRs
Why did we choose this solution?

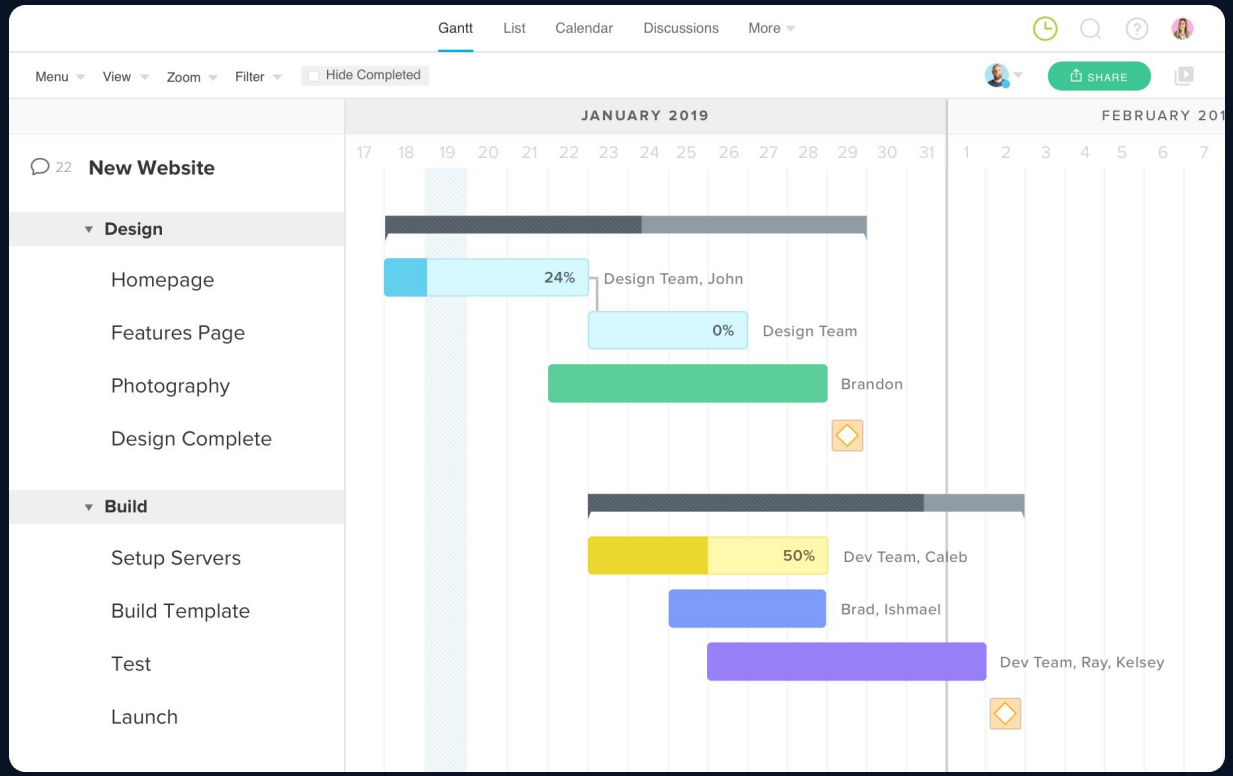
Backlog -> Ready -> In progress -> Review -> Done

Visualize the work: Kanban



Trello, GitHub Projects, GitLab Boards, Jira and Asana implement the same visual workflow.

Plan dependencies and milestones: Gantt



Useful for milestones and dependencies. Keep detailed execution in the issue board.

Turn the roadmap into small vertical slices

OUTCOME
A useful capability

EPIC
A coherent product area

STORY
A user-visible behavior

ISSUE
One reviewable change

PR
Working evidence

Prefer end-to-end value over isolated layers. Small changes are easier to test, review and recover.

The agile delivery loop

1
PLAN
one small outcome

2
BUILD
on a branch

3
REVIEW
tests + human

4
DEMO
working software

5
LEARN
reprioritize

The shortest feedback loop wins.

Our working rhythm

WEEKLY PLANNING

Choose the next outcomes and assign owners.

TWO SHORT STANDUPS

Progress, next step, blocker. Fifteen minutes.

ASYNC UPDATES

Issues and draft PRs stay current between meetings.

WEEKLY DEMO

Show working software, collect feedback, adjust.

RETROSPECTIVE

Keep, stop, start. Improve the way you work.

Definition of Ready and Definition of Done

READY

Clear user or system outcome

Acceptance criteria

Known dependencies

Owner and reviewer

DONE

Code merged to main

Tests and checks pass

Documentation updated

Deployed or demonstrable

Done means evidence, not confidence.

AI changed the speed - not accountability

BEFORE AI

Typing, searching and boilerplate were slow.

The bottleneck was producing code.

WITH AI

Prototypes arrive quickly.

The bottleneck moves to context, review, testing and integration.

Higher throughput makes project discipline more important, not less.

AI-assisted development loop

1 DEFINE
Issue + acceptance criteria

2 PLAN
Ask AI for options and risks

3 GENERATE
One small patch

4 VERIFY
Tests, lint, runtime behavior

5 INSPECT
Read the diff and dependencies

6 REVIEW
Human approval before merge

Prompting is part of the work. Validation is the work.

Guardrails for vibe coding

SMALL DIFFS

Keep generated changes reviewable.

NO SECRETS

Never paste keys, credentials or private data.

TEST BEFORE TRUST

Run it. Break it. Check edge cases.

OWN THE DESIGN

AI proposes. The team decides.

CHECK DEPENDENCIES

Licenses, security, maintenance.

HUMAN OWNS THE MERGE

The reviewer is accountable.

GIT-FIRST DELIVERY

The repository is the shared workspace

Code, configuration, documentation, decisions and evidence.

Why the project lives in Git

TRACEABLE

Who changed what, when and why

REVIEWABLE

Every change can be discussed before merge

REVERSIBLE

Return to a known good state

REPRODUCIBLE

Code, config and deployment evolve together

Do not commit secrets, credentials or private datasets.

One change, one traceable path

ISSUE
purpose + acceptance

BRANCH
isolated work

COMMITS
small checkpoints

PR / MR
review + CI

MAIN
merge + deploy

No direct pushes to main. Every meaningful change passes through review and automated checks.

GitHub / GitLab are the collaboration layer

REPOSITORY
shared code and history

ISSUES + BOARD
backlog, ownership, status

PR / MR + REVIEWS
feedback before integration

CI/CD + RULES
checks, protected main, deploys

Different platform, same workflow: issue -> branch -> review -> merge.

GitHub Docs | GitLab Docs

Prerequisites: Python + Git

PYTHON

Read and modify existing code

Create environments and install packages

Call APIs and work with data

Run tests and debug errors

GIT

Clone, pull and create a branch

Commit coherent changes

Open and review a PR

Resolve a basic merge conflict

AI can help you learn faster. It cannot replace understanding the diff or the runtime.

First-week delivery checklist

1. Clone the repository
2. Create the Python environment
3. Run the application and tests
4. Create a feature branch
5. Make a small documentation change
6. Open a PR, request review and merge

If you are blocked, ask early with the command, error and context.